

Emotion Detector

Team : Compute Collective

Sansshita Baskaran, Evan Sage, Garrett Delaney

Introduction

Emotion recognition is an increasingly important area in computer vision and human-computer interaction (HCI), with applications ranging from mental health monitoring to driver alertness systems and in customer service. Many existing emotion recognition systems rely on cloud-based computation, where video data is streamed to remote servers for processing. While this is effective, this approach could also introduce issues such as high latency and bandwidth usage, privacy concerns, and reduced reliability in environments with unstable or unavailable internet connectivity. This project, Emotion Detector, allows us to investigate whether real-time facial emotion recognition can be performed entirely on an edge device. By running the full computation locally, the system aims to improve privacy, reduce latency, and enable deployment in real-world settings without cloud dependence. This project focuses on building and evaluating an on-device emotion recognition system using the NVIDIA Jetson platform.

Implementation

Hardware:

The primary hardware platform is the NVIDIA Jetson Nano [1], a small and relatively low-cost single-board computer for embedded AI applications, featuring a quad-core ARM CPU and a CUDA-powered GPU to run machine learning (ML) and computer vision (CV) workloads at the edge [2]. We also needed a webcam to plug into the Jetson for real-time emotion detection. At first, Professor Stan supplied us with the Pixy2 robot vision camera intended for Arduino or Raspberry Pi. The Jetson lacked the ribbon cable port intended for the camera. There was a USB option, however it did not work as a standard USB webcam, but rather a debug device for their custom PixyMon software. Thus, we rented a Zoom Q4 USB Webcam from the school library and used that instead. We also utilized a USB keyboard, mouse, and HDMI monitor that members of the team already owned.

Software:

The primary computer vision library we used was OpenCV [3], which provides tools for image and video processing, feature detection and real-time vision applications across platforms. The library was originally developed by Intel and includes bindings to many programming languages; we utilized the Python version. For the machine learning framework we used TensorFlow [4], developed by Google for building, training, and deploying neural networks and other ML models. It supports CPUs, GPUs, TPUs, etc. and is widely used in academia and production level code. We also utilized the NVIDIA Jetson Software Development Kit (SDK),

for GPU acceleration in TensorFlow. The last major component was V4L2 (Video for Linux 2), a Linux kernel API that provides standardized interfaces for video capture and output devices, such as USB webcams.

Dataset:

The primary dataset for this project was the FER-2013 dataset [5]. It is divided into a train-test split with 28,709 images (89%) in the training set and 3,589 images (11%) in the test set. Each image consists of a 48x48 pixel grayscale image of a face, roughly centered in the photo. The images are categorized into 7 emotions: Anger, Disgust, Fear, Happy, Neutral, Sad, and Surprised. We also utilized an H5 file for the model, where other people had previously trained ML models on this FER-2013 dataset and we reused their model weights and parameters.

Results

This section summarizes our progress with the system, its functionality, and challenges. A major outcome was the successful setup and stabilization of the NVIDIA Jetson platform. Initial power issues with the original board prevented reliable operation and ultimately required hardware replacement. After resolving those issues, the NVIDIA JetPack SDK [6] was installed, GPU acceleration was verified, and overall system stability was confirmed. This established a functional foundation for edge-based AI deployment.

Once the board started working, we confirmed the working of local AI inference by running pre-trained image classification models directly on the Jetson. We verified that neural network inference could be executed entirely on device using GPU acceleration, without reliance

on cloud services. The system was also able to run multiple models for testing and comparison, demonstrating flexibility in the software stack. Following this, webcam integration was completed using OpenCV and V4L2, allowing continuous live video streaming into the Jetson.

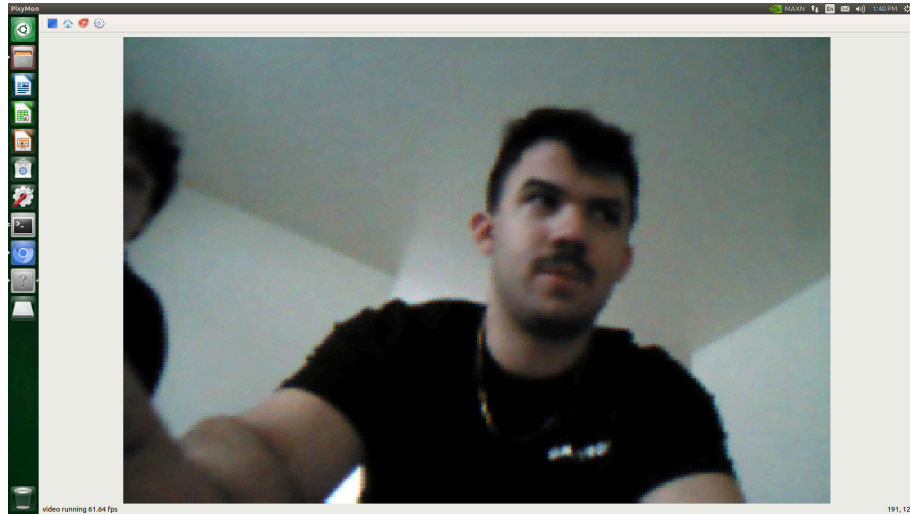


Figure 1: Jetson successfully capturing and displaying live webcam video

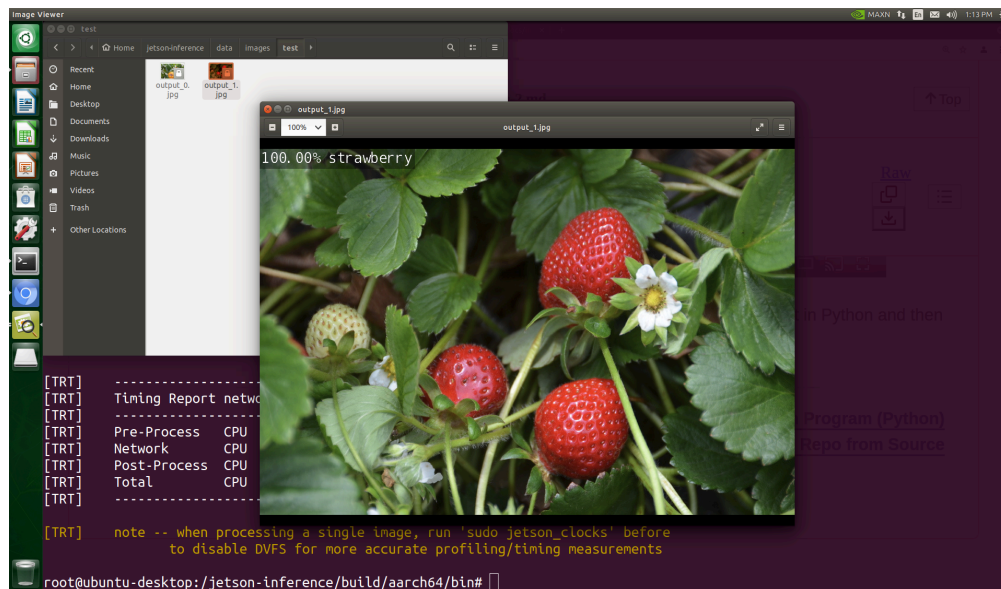


Figure 2: Jetson captioning an image of strawberries with 100% confidence

Figure 1 and Figure 2 are our initial progress pictures which is a critical step because it demonstrated a stable video pipeline, which is required for real-time emotion recognition. While this task was not emotion detection, it served as an important validation of the jetson inference pipeline, and overall software configuration, confirming that on-device inference was functioning correctly.

The final system was able to perform live facial emotion detection using the webcam feed. The final result images show faces detected in real time with bounding boxes and predicted emotions such as neutral, angry, sad, and surprised as shown in the figures below. These results show an end-to-end working pipeline, from video capture and preprocessing to on-device inference and visualization, without the use of external servers.

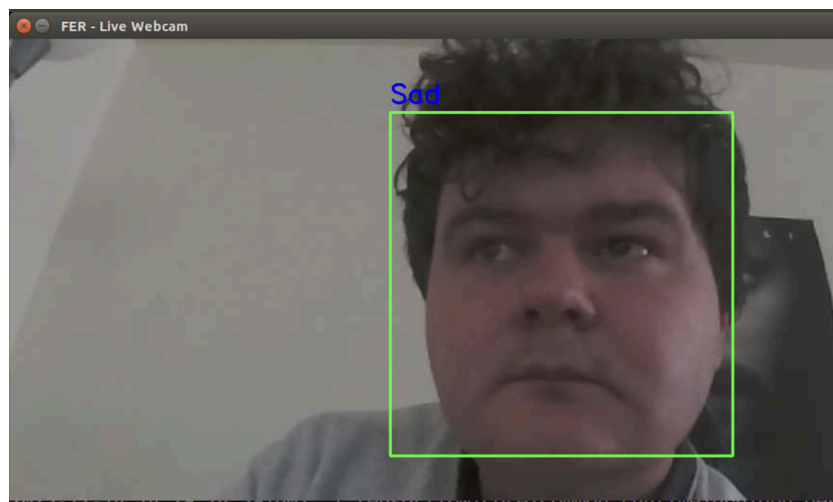


Figure 3: Picture depicting **Sad** emotion



Figure 4: Picture depicting **Angry** emotion

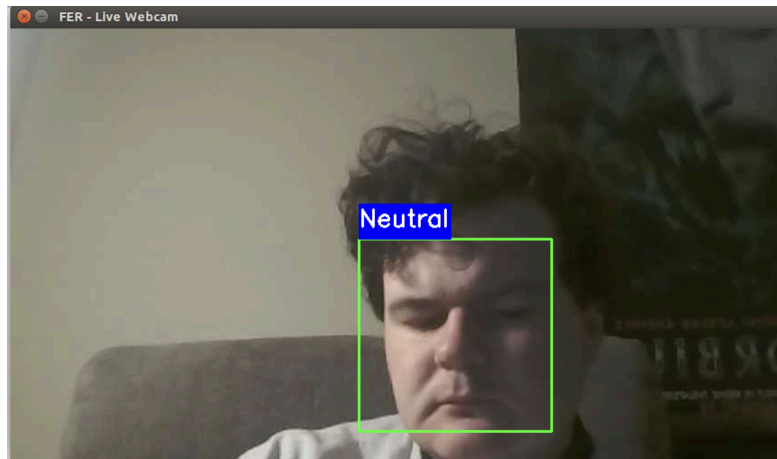


Figure 5: Picture depicting **Neutral** emotion

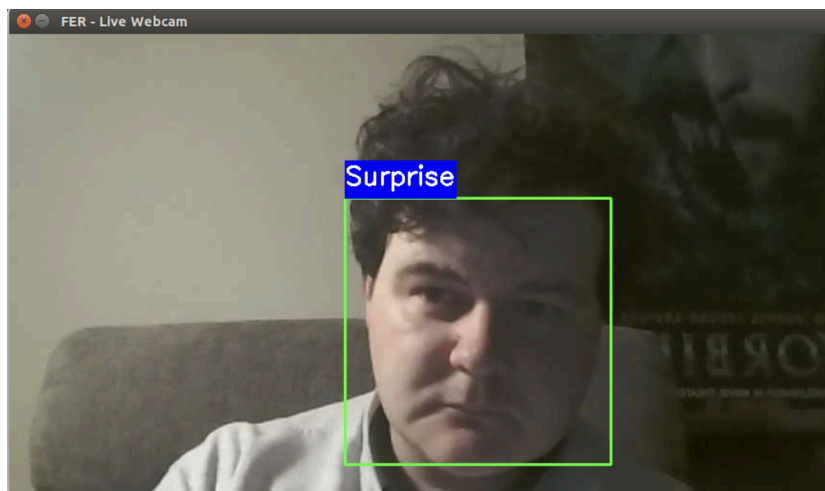


Figure 6: Picture depicting **Surprise** emotion

Despite achieving full functionality, system performance was limited. The video stream operated at approximately 5 frames per second (FPS), resulting in laggy playback and unstable predictions across consecutive frames. The model also showed a tendency to favor certain emotions, particularly surprise, while emotions such as fear and disgust were frequently overridden or misclassified. These behaviors could have been influenced by imbalance in the FER-2013 dataset, limited preprocessing, and memory constraints on the Jetson platform.

Several challenges contributed to these limitations. In addition to hardware replacement and software dependency issues, memory usage proved to be a major bottleneck. The emotion recognition models were not optimized for edge deployment, leading to high memory consumption and frequent crashes. Reinstallation was required for a few libraries such as TensorFlow and OpenCV. Additionally, the originally planned Pixy camera was incompatible with standard Linux video interfaces, and it uses its own custom software. So, we needed to replace it with a standard USB webcam.

Overall, the results demonstrate that fully on-device emotion detection is achievable on an edge AI platform, but also highlight the importance of model optimization, efficient memory management, and hardware-aware design for achieving stable real-time performance.

Conclusion

An important lesson we learned was the difference between running an image detection model on a still image in post-processing versus running on a live camera feed in real time. With still images, training is straightforward since each image is independent. You can batch, shuffle, and augment images without worrying about timing or order. With live videos, training is often still done with frames extracted from the video, but you can also use temporal information. In

terms of deployment and inference, with still images the input is static as the model only needs to process one image at a time. The latency is less critical as you can afford slower inference. If the processing is occurring in post (after the image/video has been captured) then it can take as long as it needs to, there are no time constraints. However, for the model to run in real time and display the video smoothly, the classification algorithm must be able to run in the amount of time each frame is displayed. For example, if the video runs at 30 FPS, then all image processing needs to occur within 1/30 of a second. If it takes any longer, then the system will still be processing the previous frame while the next frame comes in, and it will have to drop that next frame. The input is a continuous stream of frames, and the real-time constraints require very low-latency processing. The model efficiency is critical, and you need optimized model and hardware acceleration.

Another valuable lesson we learned is that memory management and model size are of paramount importance when running on a resource-constrained device at the edge. A certain component of TensorFlow and how we were using it had a memory leak. Thus, after a couple runs of the inference, the Jetson would crash, necessitating a system restart. Another thing is that the H5 models we utilized were trained on full size desktop computers or cloud servers. They were not optimized for the edge, and the models took up a large memory footprint. In future work, we would work more to cut this down, such as via quantization to 8- or 16-bit integers.

Something else we learned is that the Nvidia Jetson is a delicate platform and can be extremely frustrating to work with. For example, we had to get a second Jetson because the first one would not even power on. And even when we received a Jetson that would power on, it still could be a nightmare to work with. There were lots of installation and dependency issues. Sometimes this would be because the Jetson was using specific versions of libraries written by or

bundled in by Nvidia that were incompatible with the version TensorFlow required as a dependency.

References

- [1] “Jetson Nano,” NVIDIA Developer. Accessed: Dec. 15, 2025. [Online]. Available: <https://developer.nvidia.com/embedded/jetson-nano>
- [2] Boesch, “Explore Computer Vision with NVIDIA Jetson Modules,” viso.ai. Accessed: Dec. 15, 2025. [Online]. Available: <https://viso.ai/edge-ai/nvidia-jetson/>
- [3] “OpenCV,” OpenCV. Accessed: Dec. 15, 2025. [Online]. Available: <https://opencv.org/>
- [4] Google, “TensorFlow,” TensorFlow. Accessed: Dec. 15, 2025. [Online]. Available: <https://www.tensorflow.org/>
- [5] M. Sambare, “FER-2013,” Kaggle. Accessed: Dec. 15, 2025. [Online]. Available: <https://www.kaggle.com/datasets/msambare/fer2013>
- [6] “NVIDIA JetPack Software Stack,” NVIDIA Developer. Accessed: Dec. 15, 2025. [Online]. Available: <https://developer.nvidia.com/embedded/jetpack>